OPENCUA: Open Foundations for Computer-Use Agents

Xinyuan Wang [*x] **Bowen Wang** [*x] **Dunjie Lu** [*x] **Junlin Yang** [*x] **Tianbao Xie** [*x] **Junli Wang** [*x]
Jiaqi Deng [x] **Xiaole Guo** [x] **Yiheng Xu** [x] **Chen Henry Wu** [c] **Zhennan Shen** [x] **Zhuokai Li** [x] **Ryan Li** [x]
Xiaochuan Li [x] **Junda Chen** [x] **Boyuan Zheng** [x] **Peihang Li** [x] **Fangyu Lei** [x] **Ruisheng Cao** [x] **Yeqiao Fu** [x]
Dongchan Shin [x] **Martin Shin** [x] **Jiarui Hu** [x] **Yuyan Wang** [x] **Jixuan Chen** [x] **Yuxiao Ye** [x] **Danyang Zhang** [x]
Dikang Du [m] **Hao Hu** [m] **Huarong Chen** [m] **Zaida Zhou** [m] **Yipu Wang** [m] **Heng Wang** [m]
Diyi Yang [s] **Victor Zhong** [w] **Flood Sung** [m] **Y.Charles** [m] **Zhilin Yang** [m] **Tao Yu** [†x]

[x] XLANG Lab, University of Hong Kong [m] Moonshot AI
[s] Stanford University [w] University of Waterloo [c] Carnegie Mellon University

OpenCUA Homepage (Tool, Model, Data): `https://opencua.xlang.ai`

Abstract

Vision-language models have demonstrated impressive capabilities as computer-use agents (CUAs) capable of automating diverse computer tasks. As their commercial potential grows, critical details of the most capable CUA systems remain closed. As these agents will increasingly mediate digital interactions and execute consequential decisions on our behalf, the research community needs access to open CUA frameworks to study their capabilities, limitations, and risks. To bridge this gap, we propose OPENCUA, a comprehensive open-source framework for scaling CUA data and foundation models. Our framework consists of: (1) an annotation infrastructure that seamlessly captures human computer-use demonstrations; (2) AGENTNET, the first large-scale computer-use task dataset spanning 3 operating systems and 200+ applications and websites; (3) a scalable pipeline that transforms demonstrations into state–action pairs with reflective long Chain-of-Thought reasoning that sustain robust performance gains as data scales. Our end-to-end agent models demonstrate strong performance across CUA benchmarks. In particular, OPENCUA-32B achieves an average success rate of **34.8%** on **OSWorld-Verified**, establishing a new state-of-the-art (SOTA) among open-source models and surpassing OpenAI CUA (GPT-4o). Further analysis confirms that our approach generalizes well across domains and benefits significantly from increased test-time computation. We release our annotation tool, datasets, code, and models to build open foundations for further CUA research: `https://opencua.xlang.ai`

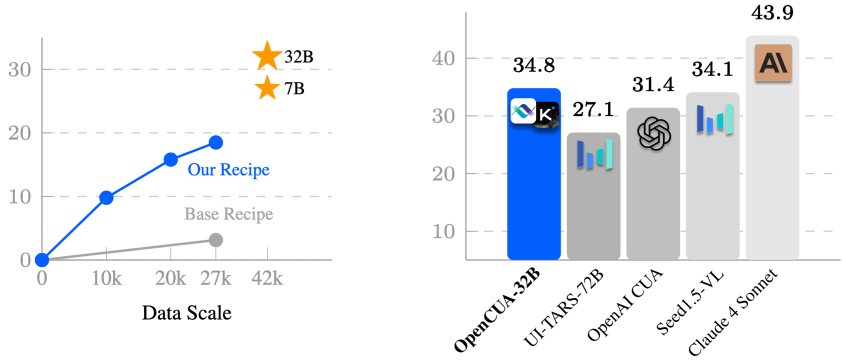

Figure 1: **Left**: OpenCUA recipe helps performance scaling with data size and model size. **Right**: Performance on **OSWorld-Verified** [42]: OpenCUA-32B outperforms current open-source models and is comparable with OpenAI CUA (GPT-4o based) [28].

[*]Equal contribution. †Corresponding authors.

Preprint. Under review.

Figure 2: Overview of the OPENCUA framework. Top left: AGENTNET TOOL captures user interactions across OSes with screen videos and action flows. Top right: Raw demos are processed into state-action trajectories with reasoning and history. Bottom right: AGENTNET Dataset and Bench provide diverse tasks and offline evaluation with gold-standard actions. Bottom left: OPENCUA models are trained and able to execute in realistic environments.

1 Introduction

Computer-use agents (CUAs), powered by vision-language models (VLMs), aim to autonomously complete computer tasks and have great potential in facilitating daily and professional work flows. Despite their growing role in high-stakes decision-making, critical details including training data, architectures, and development processes about how state-of-the-art CUA systems are built remain closed and proprietary [3, 4, 5, 17, 28]. As the lack of transparency limits technical advancements and raises safety concerns [31, 38, 11], the research community needs truly open CUA frameworks to study their capabilities, limitations, and risks.

However, current open-source attempts in CUA face significant challenges that impede progress. Firstly, there is no open-source scalable infrastructure for collecting diverse large-scale computer-use data - a complex requirement that involves the real-time capture of user interactions and state information, followed by transformation into agent-executable trajectories. Secondly, existing open-source graphical user interface (GUI) datasets remain limited in scope and scale due to the complexity and high cost of data collection; they either focus on specific domains (grounding [10, 9, 41, 18], mobile [30, 24], or web [14, 12]) or lack sufficient diversity for general computer-use applications. Furthermore, many CUA works provide insufficient details about their modeling strategies and training recipes, making replication difficult even with access to their collected data. These limitations collectively hinder advances in general-purpose CUAs and restrict a meaningful exploration of their scalability, generalizability, and potential learning approaches.

To address these challenges, we introduce OPENCUA, a fully open-source framework to scale the CUA data and the foundation models (Figure 2). To address infrastructure challenges, we first develop a user-friendly, cross-OS computer task annotation application, AGENTNET TOOL that can be installed on personal computers to seamlessly record natural human demonstrations and corresponding computer states, without disrupting the user's work flow (Figure 2 top left). We then collect the AGENTNET dataset, including more than 20K open-domain computer task trajectories spanning over 100 applications and 200 websites across Windows, macOS, and Ubuntu (Figure 2 top right). This dataset authentically captures the complexity of human behaviors and environmental dynamics from users' personal computing environments. Furthermore, given that online CUA benchmarks such as OSWorld [42] require substantial environment setup effort and runtime resources, we curated AGENTNETBENCH based on our collected human demonstrations (Figure 2 bottom right). This offline benchmark provides multiple gold-standard actions per step, efficiently approximating online metrics to dramatically accelerate agent evaluation and development.

Critical to our OPENCUA framework is our (1) data processing pipeline and (2) novel modeling and training recipe for constructing CUA training data from human demonstrations. We first introduce an

action discretization pipeline that converts raw human demonstrations, which typically consist of videos and high-frequency, redundant keyboard/mouse actions, into state-action pairs feasible for vision language model training. Despite this, we observe that training on state-action pairs alone yields limited performance gains even as the dataset size scales (see Figure 1 Left). Our first key insight is that scaling agent capabilities requires augmenting these trajectories with reflective long Chain-of-Thought (CoT) reasoning. We propose a reflective CoT synthesis method that explicitly injects planning, memory, and reflection into the per-step reasoning process through natural language "inner monologue" (Section 3.1). Different from previous work, our reasoning traces are notably more detailed and contain reflection thoughts that help the agent detect and recover from errors. Moreover, we identify key modeling details that improve agent performance (Section 3.2), such as multi-image history. Finally, we show that carefully designing training data mixtures—including diverse reasoning and general text—is beneficial for computer-use agent training (Section 3.3).

Built upon our methodology, we developed strong computer-use agent models through supervised fine-tuning (SFT) (Figure 2 bottom left). Our results show that our approach enables robust performance scaling with increased data size (Section 4.2). Our model, OPENCUA-32B, achieves a success rate of 34.8% (100 step) on OSWorld-Verified [42, 45], establishing a new state-of-the-art among the open-source models, even outperforming the proprietary OpenAI CUA (GPT-4o based) [28]. We did extensive experiments and analysis on various model structures and data scales in Section 5. Because of the diversity and coverage of our training data, our models show strong cross-domain generalization. Our agent models also show promising scalability with increased test-time compute, such as increased number of steps and larger Pass@n evaluation. We also did additional experiments, including grounding and robustness analysis. Finally, we also provide detailed ablations to justify the important design choices in our method and training recipe (Section 5). We open-source the complete suite of our OPENCUA framework, including the annotation tool, collected datasets, code, benchmark and models, providing open foundations for further CUA research.

2 AGENTNET Collection

OPENCUA aims to scale desktop computer-use data across diverse computer environments and user scenarios. We prioritize collecting demonstrations that follow natural user behavior, imposing the least additional constraints on how users interact with their computers to improve the scalability of data collection. To this end, we developed AGENTNET TOOL and collected AGENTNET dataset, the first large-scale desktop agent task dataset.

2.1 Task Definition

Human Action	Action Description	Agent Action
Click	Click at a specific position	click(x, y, button)
Middle Click	Middle click at a specific position	middleClick(x, y)
Double Click	Double click at a specific position	doubleClick(x, y, button)
Triple Click	Triple click at a specific position	tripleClick (x, y, button)
Mouse Move	Move mouse to a specific position	moveTo(x, y)
Drag	Drag mouse from one position to another	dragTo(x, y)
Scroll	Scroll vertically or horizontally	scroll(dx, dy) / hscroll(dx, dy)
Type	Type a string of text	write(text)
Press	Press a specific key	press(key)
Hotkey	Perform a combination of keys	hotkey(key1, key2)
Terminate	End the task with success or failure	terminate(`success' or `fail')

Table 1: Overview of Human Actions and Corresponding Agent Action Functions

We model the agent's decision-making process – iterative observation of the computer state followed by action prediction – as a state-action transition trajectory $(\text{or } h = \{s_0, a_0\}, \{s_1, a_1\}, \ldots, \{s_T, a_T\})$. Given a task language instruction I and initial state s_0, the agent sequentially predicts a action a_i until goal state s_t and performs the termination action, $a_n : P(a_i | I, s_0, a_0, \ldots, s_i)$.

An important design choice in building computer-use agent is to convert computer state s_i into model observation. In this work, we follow the recent trend of building pure vision-based computer agents [29, 48, 41] and use the screenshot of the computer as the observation for the agent. We use human

computer-use actions, including keyboard and mouse movements, as the action space. To ensure the action space is applicable across various operating systems, we select a subset of PyAutoGUI actions and augment them with several necessary agent actions including the `success' and `fail' termination actions. The complete action space and its parameters are listed in Table 1.

2.2 AGENTNET TOOL

Efficient and accurate annotation is essential for collecting high-quality computer-use agent data, yet no existing tools support natural, cross-platform task recording by non-technical users. To address this, we developed a user-friendly annotation tool that streamlines the collection and verification of computer-use demonstrations (Figure 3), runs on annotators' personal computers and records demonstrations in the background, capturing: (1) screen videos, (2) mouse and keyboard signals, and (3) accessibility trees (Axtree). These data are then processed into state-action trajectories (see details below in Section 2.2), allowing annotators to review, edit, and submit demonstrations along with task instructions describing the overall goal. Former works require the annotators to demonstrate "gold" trajectories with all-correct steps, but this actually limits model's capability to detect and recover from errors. We believe that annotation error is not all bad, as long as we can identify and utilize them (see Section 3.1), so we relax the requirement of all correct actions. Our implementation leverages several established tools: mouse and keyboard input tracking is based on DuckTrack [34] and OpenAdapt [17]; screen recording utilizes OBS Studio [26]; and accessibility tree (Axtree) parsing follows the OSWorld framework [42]. Additional implementation details can be found in Appendix C.3.1.

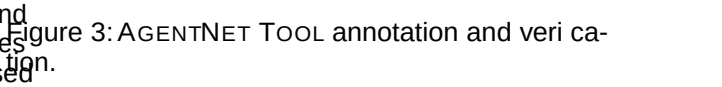

Figure 3: AGENTNET TOOL annotation and verification.

Annotation pipeline We designed our data collection with two key goals: diversity and complexity. Annotators were provided a curated list of around 200 applications and websites spanning various domains and were encouraged to demonstrate complex workflows involving professional features or multi-app interactions. Tasks were required to have more than 15 steps; those with <5 steps were rejected. To ensure wide coverage and real-world authenticity, we recruited annotators from both crowd-sourcing platforms and annotation companies. All annotators signed consent forms, and we use a multi-layer privacy protection mechanism to safeguard user data (Appendix C.3.2). To study model generalization, we split data into Windows/macOS and Ubuntu, ensuring no overlap with OSWorld tasks to prevent data leakage. All tasks were manually verified and labeled as rejected, ok, good, or excellent based on goal clarity, diversity, and complexity. Other annotation details are provided in Appendix C.2.

Constructing compact state-action trajectories Raw demonstrations consist of high-frequency screen recordings and fine-grained interaction signals (mouse movements, clicks, scrolls, key presses). A typical task can produce thousands of low-level actions that are too dense and inefficient for training. To address this challenge, we developed techniques including action reduction and state-action matching to construct compact state-action pairs (s_i, a_i). (1) Action reduction: We developed a rule-based method to compress and reduce these dense action signals into a smaller set of meaningful actions while preserving essential action information. We first compress atomic signals into higher-level operations. Mouse move events are treated as preconditions for clicks or drags, and only their start and end positions are retained. Scrolls are merged into single-directional actions with accumulated wheel counts. Consecutive key presses are merged into text input strings, while modifier combinations (e.g., CTRL+C) are abstracted into hotkey actions. We also combine common multistep gestures such as drags or double-clicks. This process yields a streamlined action sequence aligned with the pyautogui action space, as shown in Table 2. (2) State-action matching To pair each action a_i with a representative state s_i, we extract keyframes from the screen recording that capture the system state immediately before the action occurs. However, naively aligning keyframes to action timestamps of mouse clicks risks leaking future information; e.g., the mouse may already be positioned over a button, making the prediction trivial. To address this challenge, for mouse

Table 2: Comparison between OPENCUA and Other GUI Datasets

Dataset	Tasks	Avg. Step	Env. Type	Personalized Env.	Human Traj.	Dom/ AxTree	Video	Inner Monologue
AndroidControl[22]	15283	5.5	Mobile	7	3	3	7	Short
AMEX[8]	2991	11.9	Mobile	7	3	7	7	7
AitW[30]	2346	8.1	Mobile	7	3	3	7	7
AitZ[52]	1987	6.0	Mobile	7	3	7	7	Short
GUI Odyssey[24]	7735	15.3	Mobile	7	3	7	7	7
WonderBread[37]	598	8.4	Web	7	3	3	3	7
AgentTrek[46]	10398	12.1	Web	7	7	3	3	Short
Mind2Web[12]	2350	7.3	Web	7	3	3	7	7
GUIAct[9]	2482	6.7	Web	7	3	3	7	7
AgentNet	20631[2]	17.2	Desktop	3	3	3	3	Long

clicks, we backtrack to the beginning of the mouse's pre-movement phase and search backward to find the last visually distinct frame. After the final action, we append a terminal frame along with a corresponding termination action.

2.3 AGENTNET Statistics

Our dataset consists of 20,631 human-annotated computer-use tasks, including 12K from Windows, 5K from macOS and 3K from Ubuntu, with screen resolutions ranging from 720p to 4K. Each trajectory averages 17.2 steps, reflecting task complexity. As shown in Figure 4, the data spans over 140 applications and 190 websites, often involving multi-app workflows, professional tools, and uncommon features. Compared to prior GUI datasets (Table 2), AGENTNET is the first desktop trajectory-level dataset that is realistic, complex, diverse, and multimodal. Full statistics are provided in Appendix C.1.

3 Training Computer-Use Agent Model

Our AGENTNET consist of task instructions I and state-action (s_i, a_i) trajectories. However, we find that directly fine-tuning vision-language action (VLA) models on our 27K trajectories leads to poor performance (4.4% success rate on OSWorld [42], shown in Figure 1 Left "Base Recipe"). This section presents our modeling and training recipe to enable scalable training of computer-use agent models, including a novel reasoning augmentation, context encoding, and data mixtures techniques.

Figure 4: Domain distribution of tasks in AgentNet dataset

3.1 Synthesizing Reflective Long CoT Reasoning

Consistent with prior works [51, 48, 29], we find natural language reasoning crucial for generalizable computer-use foundation models, helping CUAs internalize cognitive capabilities. We propose a multi-stage CoT framework synthesizing structured reasoning per state-action pair (s_i, a_i). Inspired by Aguvis [48], our structured CoT includes three reasoning levels. The hierarchy begins with L3, contextual observation capturing salient visual and textual elements. L2 next provides reflective reasoning analyzing state transitions, recalling previous steps, correcting errors, and planning subsequent actions. Finally, the model predicts L1, a concise executable action grounded in prior perception and thought. This L3 → L2 → L1 structure mirrors perceptual-to-agentic decision flow, equipping the model with coherent, interpretable reasoning trajectories that enhance generalization and robustness.

Reflection augmentation for L2 reasoning Incorrect or redundant annotations in human demonstrations are not all bad, as long as we can identify and use them to teach the identification and

[2] A total of 41,428 trajectories were used to train OPENCUA-7B and OPENCUA-32B, and 27,804 trajectories were used for training OPENCUA-QWEN2-7B and OPENCUA-A3B. Moonshot AI annotated the Ubuntu subset and generously agreed to release 3K of the annotated trajectories to the public.

Figure 5: Re ective long CoT synthesis pipeline: generator and re ector iteratively generate and verify the reasoning components between the observation and ground-truth actions.

Figure 6: Re ective Long CoT Example: Before predicting the actual action, the model rst reason according to the history and current action with re ection, memory, plan&reasoning and prediction in the CoT. The model identi es the former mistake and correct it in the later steps.

correction of model errors. Therefore, we designed a reflector to identify errors and generate re ection reasoning for each step. Our CoT synthesis framework extends the pipeline of AgentTrek [48] and ActRe [50] by equipping the "Thought" with more comprehensive agent components, especially state-transit perception and re ection, and minimizing hallucination. As shown in Figure 5, our CoT synthesis pipeline consists of three components: a reflector , a generator , and a summarizer. The reflector inspects each step for correctness and redundancy by comparing screenshots before and after the action, examining the correctness of the action code itself and the generated CoT, expecially whether the "Action" aligns with the screenshot and code. When the step is incorrect or redundant, the reflector will ellaborate reason and this step will be ignored during training. If the step is correct, the reflector will explain the differences the actions brings to the before and after state. The generator conditions on the full agent context—previous re ections, action history, task goal, screenshots, and action code—to generate structured CoT. To help the model ground coordinate-related actions more accurately, we incorporate visual cues: a red marker on the mouse action coordinate and a zoomed-in image patch (inspired by SE-GUI [32]). Finally, the summarizer re nes vague user-written goals into more precise and aligned task objectives, and scores each trajectory for alignment, ef ciency, and dif culty. Our method produces rich and meaningful CoTs that signi cantly improve model reasoning and planning. We use claude-3-7-sonnet-20250219 as the base model for synthesizing the three components. The re ection helps agent model identify former errors and adjust future plan to make the task back to the right track. An example of error identi cation and correction in evaluation can be seen in Section F. Ablations in Section 5 demonstrate that this module is a important driver of performance gains.

3.2 Context Encoding and Test-Time Reasoning

For end-to-end agent models, history encoding plays a critical role in reasoning and planning.

Context encoding (1) **Textual history**: We propose a structured inner monologue framework for textual history representation. Specifically, we adopt a dialogue-style history format representing the model's responses and corresponding computer screenshots. Following AgentS2 [3], we use L1 CoT (Action) to represent previous steps because it is more token-efficient and allows longer history windows without truncation. Moreover, our inner monologue includes memory components, further compensating for the absence of richer CoT in earlier steps. See Section 5 for ablations on history representation. (2) **Visual history**: Multi-image screenshot history is essential for agent model performance because screenshots are lossless visual representation of history, providing more grounded context than textual summaries. However, including more images also increases input length and reduces training efficiency. By default, we use three screenshots as visual representation, as our experiments show that this achieves a balance between performance and efficiency (see Figure 11).

Test-time reasoning format While the model is trained with a mixture of CoT levels, we adopt the L2 CoT format at inference time due to its richer reasoning content, which enhances the model's ability to reflect, plan, and reason. As shown in our ablation studies (Section 5), L2 CoT significantly improves test-time performance scalability—Pass@success rates on OSWorld increase markedly over Pass@1. In contrast, models lacking this reasoning augmentation exhibit limited scalability, highlighting the importance of strong reasoning signals at inference time.

3.3 Training Data Mixtures

CoT data mixture: As we mentioned in the Section 3.1, our structured inner monologue contains three levels of CoT: L1 (Action), L2 (Thought + Action), and L3 (Observation + Thought + Action), each encoding complementary information for agent decision-making but has different conceptual information. L1 CoT has direct connection to the actual action; while there is helpful screenshot perception information in the L3 CoT, some irrelevant elements may also be described; L2 CoT contains planning and prediction that directly affect the predicted action in L1. Therefore, we propose to train the model with a mixture of all three levels of CoT to reinforce this different levels of connection. Data example of L1, L2, and L3 can be seen in Appendix G. We verify this design choice with ablations in Section 5.

Mixture of grounding, planning, and general SFT data: A general-purpose computer-use agent foundation model should be capable of both solving complex computer-use tasks and performing general reasoning grounded in world knowledge. To achieve this, we train on a mixture of data types that span both computer-use and general vision-language domains.

For grounding, we initialize the model using existing datasets such as ShareGPT4V [23], UI-TARS [15], and 189K bounding-box samples parsed from collected AXTree structures. For planning and reasoning, we include a diverse mix of Ubuntu and Windows/macOS demonstrations as well as task-instruction-augmented samples (Section 3.1). To enhance generalization and reasoning ability, we additionally incorporate general supervised fine-tuning (SFT) data from the Kimi Team [33]. The general text data covering domains such as instruction following, mathematical reasoning, and long-context understanding. The general vision data includes domains such as OCR and vision QA data. This mixture ensures both GUI grounding and high-level reasoning capabilities across domains. Although these general data are not directly related to GUI environments, we find that mixing them improves the model's overall agentic performance. We present ablation results on this effect in Section 5.

CUA Training Strategies Depending on computing budget, dataset availability, and the target model—e.g., building a specialized computer-use agent or a general-purpose VLM with agentic capabilities—different training strategies may be adopted. Following AgentS2 [3], which proposes a two-stage curriculum training (Stage 1 for grounding and Stage 2 for planning and reasoning), we further explore three strategies:

1. **Stage 2 only**: When training resources are limited and the focus is on computer-use agent data, we aim to adapt a general open-source VLM into a specialized CUA. To preserve general instruction-following ability, we use a training mix of 70% CUA data (with a planning-to-grounding ratio of

4:1) and 30% general SFT data. We ne-tune Qwen2-VL with 30B tokens and Kimi-VL-A3B with 20B tokens. Both models exhibit strong improvements on CUA tasks (see Table 10).

2. Stage 1 + Stage 2:With more resources and diverse data, a staged approach yields better performance. In Stage 1, we enhance grounding and understanding using grounding trajectories, tutorial-style demonstrations, state-transition caption data, general vision language tasks, and general text SFT data. We trained Qwen2.5-VL-32B on this mixture for 40B tokens. In Stage 2, we shift focus to CUA planning, using 45% planning, 20% grounding, and the rest general data. This results in OPENCUA-32B, which achieves substantial gains in both grounding and planning benchmarks (Table 3, Figure 7).

3. Joint training: To build a general-purpose VLM with strong CUA abilities, we adopt joint training across domains with balanced data mixing. Given the complexity of multi-image trajectory data, we train these samples for 3 epochs to ensure suf cient learning. Based on Qwen2.5-VL-7B, we train a model on 200B tokens budget, maintaining a data ratio of 20% planning, 20% grounding, and 60% general. The resulting model, OPENCUA-7B, achieves state-of-the-art performance among 7B-scale open-source CUAs, achieving 27.3% success rate on OSWorld Online Evaluation Platform.

4 Experiments

4.1 Experimental Setup

Models Our experiments are conducted on multiple open-sourced vision-language models: Kimi-VL-A3B [33], Qwen2-VL-7B-Instruct [36], Qwen2.5-VL-7B-Instruct [6], and Qwen2.5-VL-32B-Instruct [6][3]. Kimi-VL-A3B adopts a Mixture-of-Experts (MoE) architecture with 16B total parameters and 3B active during training and inference. It demonstrates moderate capability as a computer-use agent, including grounding and planning. Qwen2-VL and Qwen2.5-VL are general-purpose vision-language models, with Qwen2.5-VL exhibiting enhanced digital agent capabilities and expertise in high-resolution understanding. We conduct supervised ne-tuning (SFT) on these models and obtain our OpenCUA model variants: OPENCUA-A3B, OPENCUA-QWEN2-7B, OPENCUA-7B, and OPENCUA-32B.

Evaluation We evaluated our models on online evaluation benchmarks, of ine agent evaluation benchmark, and GUI grounding benchmarks.

1. Online agent evaluation: (1) OSWorld-Veri ed : OSWorld [42] originally curated 369 human-crafted tasks covering a wide range of applications, each accompanied by its own environment setup and evaluation script. The OSWorld team has now veri ed every task, xing those that were infeasible to test because of outdated dependencies, evaluation errors, or ambiguous instructions, and has released the improved benchmark as OSWorld-Verified [45][4]. Our results are obtained through the public evaluation by the OSWorld Team on AWS infrastructure. The results are reported in Table 3. (2) Windows Agent Arena (WAA) [7] contains 154 Windows-centric tasks, spanning native Windows applications as well as several open-source programs also featured in OSWorld. It can re ect the agent's online performance on Windows system.

   Evaluation setting: The resolution of the systems is 1920 1080. 4 clock tasks in WAA are dropped due to API and system image limitation. We adopt the L2 CoT format (Thought + Action) for all models, following results of our ablation in Section 5. Temperature is set to 0 for deterministic decoding during evaluation. The results in OSWorld-Veri ed are the average of 3 runs.

2. Of ine agent evaluation: AGENTNETBENCH includes 100 representative held-out tasks covering diverse domains on Windows and macOS. The introduction and details of AGENTNETBENCH are in Appendix B. We also validated its correlation with online benchmark results in Figure 14.

[3] To align with the training infrastructure of the Kimi Team, we adopt the same chat template and tokenizer as Kimi-VL-A3B. M-RoPE in Qwen models is not implemented; we use 1D RoPE [32] instead.

[4] OSWorld-Veri ed Leaderboard: https://os-world.github.io/ OSWorld-Veri ed blog: https://xlang.ai/blog/osworld-verified

3. GUI grounding evaluation: We evaluate our model's GUI grounding ability, the ability to map natural language instructions to specific actions within graphical user interfaces on three benchmarks: OSWorld-G [44], Screenspot-V2 [40] and Screenspot-Pro [21]. OSWorld-G has 564 samples that systematically cover text matching, element recognition, layout understanding and fine-grained manipulation, with annotations for the element types required to solve each task. Screenspot-V2 includes screenshots from three platforms: mobile, desktop, and web. Screenspot-Pro focuses on high-resolution desktop environments, especially in professional settings.

Training settings. All models are trained on the Kimi Team's infrastructure with the Megatron framework and DeepSpeed (ZeRO-3). We employ three training strategies:

1. Stage-2 only. OpenCUA-Qwen2-7B and OpenCUA-A3B share a configuration of sequence length 32,768, learning-rate 2×10^{-5}, weight-decay 0.1, and global batch size 384 (512 in ablations) on 96× A100 GPUs. They are trained on 18k Win&macOS + 10k Ubuntu trajectories. OpenCUA-Qwen2-7B runs for 3,400 steps (about 45 h) after a 400-step grounding warm-up; OpenCUA-A3B runs for 2,000 steps (about 10 h).
2. Stage-1 + Stage-2. OpenCUA-32B is first pretrained on 35B tokens of general text, vision, and grounding data (batch 3,584, LR 1×10^{-5}, 224× A100). We take the step-1200 checkpoint. Stage-2 then continues for 60B tokens on trajectory + general + grounding data (batch 512, LR 2.5×10^{-5}, 128× A100) using 18k Win&macOS + 20k Ubuntu trajectories. The final model corresponds to step 4,700.
3. Joint training. OpenCUA-7B is trained end-to-end on the full data mixture for 200B tokens (18k Win&macOS + 20k Ubuntu trajectories) with batch 512, peak LR 2.5×10^{-5} (min LR 3×10^{-6}), decay tokens 200B, on 128 A100 for eight days. The best checkpoint is at step 14,600.

4.2 Main Results

Table 3: OSWorld-Verified results. OpenCUA-32B achieves the best performance among all open-source models with an average success rate of 34.8%, outperforming prior baselines by large margins. It also closes the gap to proprietary agents, even surpassing OpenAI CUA. This demonstrates the scalability and strength of our OpenCUA training pipeline. (The OpenCUA-7B and OpenCUA-32B are evaluated 3 times independently Tao: then why not add Confidence intervals in the table? to directly show this. The details of each run and the Pass@3 evaluation results are shown in Appendix Table 10)

Model	15 Steps	50 Steps	100 Steps
Proprietary			
OpenAI CUA [28]	26.0	31.3	31.4
Seed1.5-VL [17]	27.9	-	34.1
Claude 3.7 Sonnet [5]	27.1	35.8	35.9
Claude 4 Sonnet [4]	31.2	43.9	41.5
Open-Source			
Qwen2.5-VL-32B-Instruct [6]	3.0	-	3.9
Qwen2.5-VL-72B-Instruct [6]	4.4	-	5.0
Kimi-VL-A3B [33]	9.7	-	10.3
UI-TARS-72B-DPO [29]	24.0	25.8	27.1
UI-TARS-1.5-7B [29]	24.5	27.3	27.4
OpenCUA-7B (Ours)	24.3	27.9	26.6
OpenCUA-32B (Ours)	29.7	34.1	34.8

Online agent evaluation. Table 3 lists success rates of end-to-end agents for 15-, 50- and 100-step budgets on OSWorld-Verified. The details of each run, Pass@3 evaluation success rate, and the results of OpenCUA-A3B and OpenCUA-7B are demonstrated in Appendix Table 10. Besides quantitative metrics, we also provide a qualitative case study in Appendix F, which shows that our agent can recognize and correct earlier mistakes during long-horizon execution, ultimately enabling more reliable task completion.

1. Proprietary models still lead, but the gap is closing. Claude 4 Sonnet achieves 43.9 % at 100 steps, followed by Claude 3.7 Sonnet at 35.9 %. OPENCUA-32B reaches 34.8 % in 100 steps, establishing a new state-of-the-art among open-source systems, surpassing the GPT-4o-based OpenAI CUA (31.4 %) and further closing the gap to Claude models.
2. OpenCUA method applies to models with different architectures and sizes. We apply our method on 4 model architectures, including Kimi-VL-A3B [33], Qwen2-VL-7B [36], Qwen2.5-VL-7B [6] and Qwen2.5-VL-32B [6], covering MoE and dense structures and activation parameter number from 3B to 32B. As the results in Appendix 10, all the models' performance improves by a large margin upon the base models. In con rming that our data pipeline and training recipe scale effectively with model capacity. Furthermore, their performance scales with the model size - OPENCUA-32B consistently outperforms OPENCUA-7B under every step budget.
3. Effect of step limit. Most agents pro t from a larger step budget from 15 to 50 steps, yet the gain from 50 to 100 steps is often smaller than that from 15 to 50 steps.
   - Claude 4 Sonnet: +12.7 % from 15 50 steps, then 2.4 % from 50 100.
   - OPENCUA-32B: +4.4 % (29.7 34.1) from 15 50 steps, and +0.7 % (34.1 34.8) from 50 100 steps.
   - Base models including Qwen2-VL and Kimi-VL: marginal gains (< 1 %), re ecting limited step scale capability of base models.

   Even top-tier agents (e.g., Claude 3.5 Sonnet) still struggle to bene t signi cantly on larger step budgets for truly long-horizon tasks. The modest gain when moving from a 50-step to a 100-step limit stems from two main factors: (i) most tasks need more than 15 but fewer than 50 GUI actions, so the extra head-room often goes unused; (ii) current models are still not good at recognising their own mistakes, recovering from errors, and deciding when to stop—hallucinations and repetitive loops frequently waste the additional steps. We did a detailed error study in Apendix E.
4. OpenCUA models achieve markedly higher Pass@n scores. As reported in Appendix 10, the Pass@3 success rate of OPENCUA-32B on OSWORLD-Veri ed jumps from 34.2% (Pass@1) to 45.6%. This large margin suggests ample headroom for future post-training, reranking or multi-agent methods. Additional analyses of test-time upper bounds and robustness are provided in Section 5.

These observations validate the effectiveness of our pipeline and highlight the remaining headroom for open-source agents to close the gap with larger proprietary models.

Of ine benchmark evaluation. AGENTNETBENCH is constructed from representative tasks in the OPENCUA dataset. It is a held-out testset including 100 task trajectories on Windows and macOS. To account for domain similarity, we group models in Zero-shot systems—those not trained on OPENCUA (Qwen2.5-VL-7B/32B/72B, Aguvis-7B, and OpenAI CUA (GPT-4o))—and Fine-tuned models, namely our OPENCUA-7B and OPENCUA-32B. As summarized in Table 4:

1. Scaling with model size. In the zero-shot group, performance scales with model size; the specialist Aguvis-7B surpasses the general-purpose Qwen2.5-VL-7B.
2. OpenAI CUA generalize well on unseen tasks. OpenAI CUA outperforms all open-source zero-shot models and approaches ne-tuned agents, particularly excelling in termin-ate detection and content-based actions.
3. Of ine benchmark is consistent with the ranking on online benchmark. The overall ordering matches the online leaderboard in Table 3: OPENCUA-32B > OpenAI CUA > OPENCUA-7B > Qwen2.5-VL models.
4. Coordinate-action performance re ects model grounding performance. OPENCUA-32B exceeds OPENCUA-7B on coordinate-based actions, aligned the grounding results in Figure 7.

GUI grounding evaluation. Figure 7 and Table 3 reveal several key ndings:

1. OPENCUA-32B ranks the rst. OPENCUA-32B is the best-performing agentic model on all mainstream GUI-grounding benchmarks. Its advantage stems from (i) the substantially larger grounding corpus used in Stage-1 training and (ii) its larger 32B parameter scale.
2. Joint-training boosts OPENCUA-7B. OPENCUA-7B also scores competitively because large-scale grounding data are injected during joint training.

Table 4: Computer-use agent performance on AGENTNETBENCH. Coord actions: click, rightClick, doubleClick, moveTo, dragTo, scroll; Content actions: write, press, hotkey; Function action: terminate

Model	Coord. SR	Content SR	Func. SR	Avg. SR
Qwen2.5-VL-7B [6]	50.7	40.8	3.1	48.0
Aguvis-7B [48]	56.7	43.3	0.0	52.4
Qwen2.5-VL-32B [6]	66.6	47.2	41.5	64.8
Qwen2.5-VL-72B [6]	67.2	52.6	50.5	67.0
OpenAI CUA [28]	71.7	57.3	80.0	73.1
OpenCUA-7B (Ours)	75.4	46.4	53.6	71.0
OpenCUA-32B (Ours)	78.7	46.0	55.2	73.2

3. Pixel-budget advantage of Qwen2.5-VL. Qwen2.5-VL based models uses a higher max pixels limit (12,845,056 vs. 829,440 for Qwen2-VL and Kimi-VL-A3B), yielding stronger results on high-resolution ScreenSpot-Pro. OPENCUA-32B achieves 55.3% on Screenspot-Pro, and OPENCUA-7B achieves 50.0%.
4. Grounding alone is not enough. Although Qwen2.5-VL-32B matches or even surpasses OPENCUA-QWEN2-7B and OPENCUA-A3B on OSWorld-G and ScreenSpot-V2, OpenCUA models achieve far higher success rates on the full OSWorld benchmark (OPENCUA-A3B 19.9% and OPENCUA-QWEN2-7B 23.0% shown in Appendix 10). This shows that solid grounding is necessary yet insufficient for realistic agentic tasks in the environment, where high-level planning and reflective reasoning ultimately drive reliable task completion.
5. Domain correlation. Because OSWorld-G [44] is collected in an Ubuntu environment, its scores correlate more closely with the online OSWorld evaluation than with ScreenSpot-Pro and Screenspot-V2.

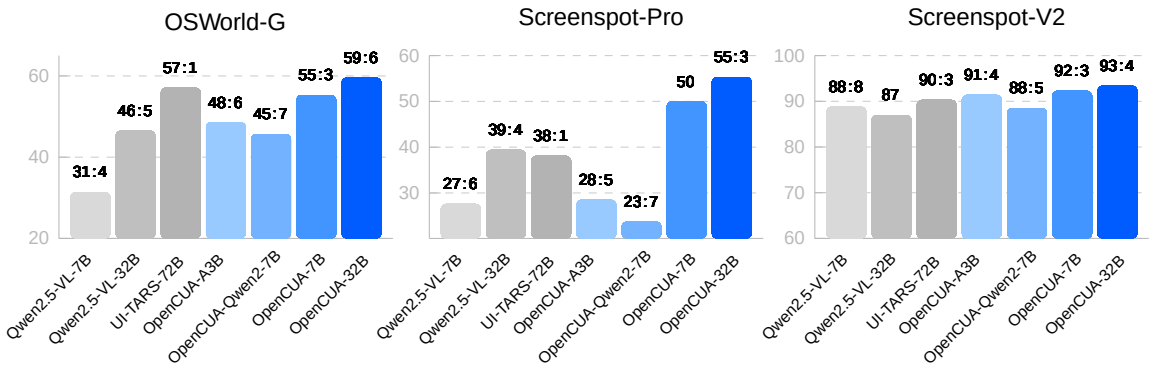

Figure 7: GUI Grounding performance of OpenCUA models compared with Qwen2.5-VL models and UI-TARS on three benchmarks.

Performance scaling with data scaling. We explore the effect of data scale on Qwen2-VL from three perspectives: cross-domain data, in-domain data, and out-of-domain data. We first investigate cross-domain data in Figure 1. Specifically, we compare three training settings: (1) 7K Ubuntu data, (2) 7K Ubuntu + 14K Win&Mac data, and (3) 10K Ubuntu + 17K Win&Mac data. On OSWorld, performance improves significantly from 9.8% to 18.5%, despite the added Win&Mac data coming from a different platform. This indicates that even out-of-domain data can substantially enhance generalization and reasoning ability, rather than causing negative transfer. To further study the impact of in-domain and out-of-domain data scale, we randomly sampled 3K, 7K, 10K trajectories from Ubuntu data and 3K, 7K 14K from Win&Mac.

As shown in Figure 8, performance scales consistently across all benchmarks with both in-domain and out-of-domain data. When increasing the Ubuntu data from 3K to 10K, the average performance improves by 72%. Scaling the Win/Mac data from 3K to 14K yields a 125% improvement on average. These results demonstrate a strong positive correlation between data quantity and agent performance, highlighting the importance of large-scale, diverse CUA data for model generalization.

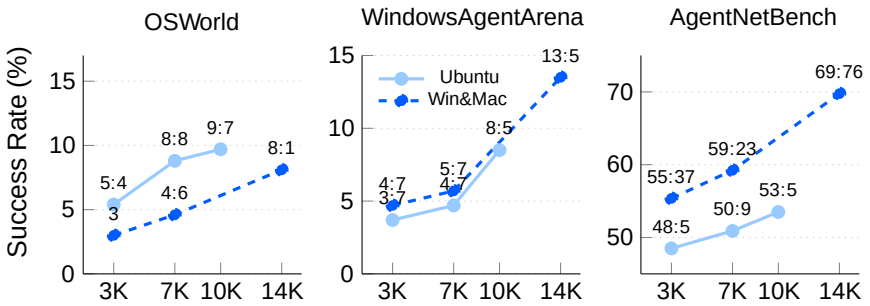

Figure 8: Scaling curves on three benchmarks as data volume from various OS domains increases.

5 Analysis

Model performance upperbound analysis by scaling test-time compute We further explore our model's performance upperbound by doing Pass@ evaluation on OSWorld. We set the temperature to 0.1 and evaluate OPENCUA-QWEN2-7B for 16 times on the budget of 15, 30 and 50 steps and calculated the pass@1,4,8,16 success rate. In Table 9, we nd: (1) There is a signi cant performance gap of our model between Pass@1 and Pass@16. On 15 step, the success rate increases from 16.9 to 34.6 (+104%), while on 50 step, the increacement is even large from 18.4 to 39.2 (+113%). (2) With larger , the performance gains from increasing the step budget become more signi cant. (3) Online benchmarks have a large vari ance. To study model robustness, we did Pass@n evaluation when temperature is 0 in Figure 5 and nd higher temperature leads to higher Pass@n performance but lower Pass@1 performance.

Figure 9: OSWorld Pass@N performance curves of OPENCUA-QWEN2-7B, temperature=0.1

We then investigated in the results and nd variance comes from these factors:

1. The agent chooses different solutions in different runs. For example, in the task “Re-open the last closed browser tab,” the agent sometimes uses Ctrl + Shift + T (only one step) and other times navigates through the history menu (needs many steps). Agents may fail on harder solutions.
2. Minor omissions or extras. In Chrome or VS Code settings, forgetting to click “Save” (or performing an additional stray click) converts a correct solution into failure.
3. Environment dynamics: Occasional CAPTCHA dialogs, machine variability, and network latency can change the interaction sequence and lead to inconsistent outcomes.

Agent model is not robust: small variance in the environment affects the task result. As illustrated in Figure 10, OPENCUA-QWEN2-7B 's OSWorld performance (Pass@N) under temperature=0 exhibits signi cant outcome divergence despite nearly identical initial states—with only minor variations (e.g., system date). The curves for Pass@16 (38.60% SR at 50 steps) and Pass@1 (20.10% SR) demonstrate a >18% absolute gap, highlighting how minimal initial perturbations propagate into starkly different trajectories. This underscores the model's sensitivity to initial conditions even in deterministic (temp=0) settings, suggesting that seemingly trivial factors (e.g., temporal context) may critically in uence multi-step reasoning.

Figure 10: OSWorld Pass@N performance curves of OPENCUA-QWEN2-7B, temperature=0

Table 5: Ablation results on OSWorld for different Chain-of-Thought (CoT) settings.

Ablation	CoT Variant	SR (%)
CoT Mixture	L2	13.1
	Mixture-CoT	18.5
Re ective Long CoT	Short-CoT	11.5
	Advanced-CoT	15.3
Test-time Reasoning Format	L1	16.9
	L2	18.5
	L3	17.6

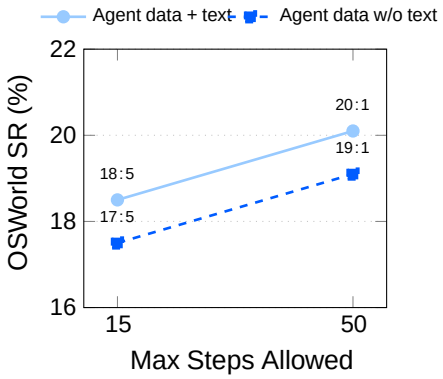

Figure 12: General text data ablation.

Cross-platform training improves generalization, even with domain differences. As shown in Figure 8, there is a consistent performance gap between models trained on different domains. Models trained on Ubuntu data perform better on OSWorld, while those trained on Windows/macOS data perform better on WindowsAgentArena and AGENTNETBENCH. This domain gap re ects the underlying differences in GUI layouts, system styles, and application behavior across platforms. OSWorld primarily focuses on applications and websites aligned with Ubuntu environments, whereas WindowsAgentArena contains several OSWorld Windows-speci c applications. Interestingly, the performance gap between training on Win&Mac data versus Ubuntu data is narrower on WAA than on OSWorld, suggesting that application-level knowledge can partially transfer across operating systems, even if interface styles differ.

L2 reasoning format achieves the best inference performance.Note that we trained the models with mixed reasoning format (L1, L2, L3, see Section 3.1). In this part, we explore which type of thinking format is the best at inference. We test OPENCUA-QWEN2-7B and OPENCUA-A3B on OSWorld in 15 steps. As in Table 5, using the L2 format, the performance is higher than L1 and L3. This result is actually different from the conclusion from previous work [48, 29] that L1 outperforms L2. We think this is because our L2 CoT has higher quality (e.g., planning and re ection), which can help the model make better decisions. On the other hand, L3 underperforms L2. By case study, we nd that when model describes the information in the screenshot, there tend to be many elements irrelevant to the task or the next action, which may mislead the model. In summary, our results show that the right choice of high-quality, non-redundant reasoning can leverage VLM's language reasoning capability to improve the agent performance.

Using a moderate number of visual history images and concise textual history yields the best trade-off between performance and ef ciency. We ablate history representation from both visual and textual perspectives. For vision, we vary the number of history screenshots (1, 3, 5) and ne-tune Qwen2-VL-7B on 7K trajectories. As the OSWorld results shown in Figure 11, using multiple screenshots substantially improves performance over single-image inputs, as GUI agents rely entirely on vision for observing state changes. However, increasing from 3 to 5 images yields marginal gains while incurring 3K more context tokens and delayed convergence, suggesting diminishing returns.

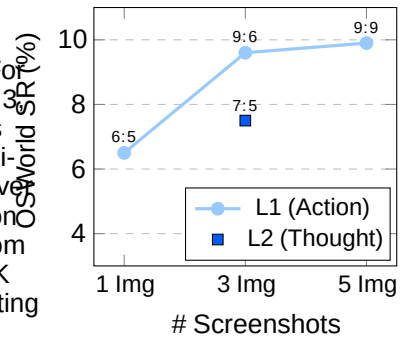

Figure 11: Effect of history representation: L1 (Action) bene ts from more screenshots, while L2 (Thought) at 3 images lags behind.

On the textual side, we compare L1 and L2 history under the same 3-image setting. In Figure 11, L2 history offers no bene t and may introduce hallucinations that distract attention, while also reducing training ef ciency. Hence, we adopt L1 CoT + 3 images as the default setting.

Training with a mixture of CoT formats outperforms using only L2 reasoning. Since our best performance is from L2 CoT inference, and L3 and L1 is lower than L2, we did an ablation of

only training the L2 data instead of the mixture of L1, L2, and L3. We use the same recipe as our OPENCUA-QWEN2-7B, but only replace the mixture CoT data with L2 data. As the OSWorld result in Table 5, the model trained on L2 data using the same steps as OPENCUA-QWEN2-7B, but the performance drops to 13.1, which is aligned with the conclusion of Aguvis [48].

General-domain text data provides a positive effect to agent performance. As we mentioned in Section 3.3, we used 35% general text data in our main experiment, so we also use the same agent data without the text data to ne-tune Qwen2-VL-7B with grounding warm-up stage for 2400 steps (approximately the same amount of agent data tokens) to ablate its in uence. According to Figure 12, the general text data slightly improves model's agentic performance. Therefore, adding text data from totally different general domains doesn't impair the agent model's performance, on the contrary, helps improve the performance. We think the reason is that the general text data may help agent model's generalization and instruction understanding.

Re ective long CoT signi cantly boosts performance by improving error correction. To understand the effect of re ective long CoT (Secion 3.1), we do an ablation study on Qwen2-VL-7B with 14K Win&Mac and 3K Ubuntu trajectories. Without re ective long CoT, the CoT reduces to that used by Aguvis [48]. In Figure 6, we see that re ective long CoT improves the performance from 11.5 to 15.3. Since the re ective reasoning focuses on error correction, we conjecture that the improvement comes from improved self-correction capability.

6 Related Work

CUA benchmarks and datasets Autonomous computer-use agents are now judged primarily through execution-level benchmarks that embed the agent inside genuine software environments rather than synthetic simulators. On the desktop side, suites such as [43, 7, 53] orchestrate multi-step work ows that span of ce productivity, source-code editing, le management, and operating-system con guration across Linux, Windows, and macOS. For the web domain, campaigns including [54, 19, 12, 49, 13] deploy agents on self-hosted or live sites with dynamic content, long navigation chains, and non-trivial information-retrieval subtasks. To support training at the necessary scale, several high-volume data pipelines have appeared: tutorial-to-trajectory conversion for browser tasks [46], cross-device grounding and action logs [18, 10, 16], plus our own collection of 27 K desktop demonstrations that pair screenshots, low-level mouse/keyboard events, and re ective chain-of-thought annotations.

CUA frameworks and models Approaches to building computer-use agents can be grouped into three broad categories. First, text-based language models operate on structured GUI metadata—such as DOM trees or accessibility labels—and issue symbolic commands; representative work ranges from early page-centric agents [25] to more recent language-only planners that still eschew raw pixels [47]. Second, vision-centric agents integrate screen imagery. Some focus on grounding—learning to associate natural-language references with bounding boxes or coordinates [16, 40, 14]—while others pursue end-to-end policies that translate full screenshots directly into action sequences [48, 29, 28, 3, 33, 17]. Third, agent frameworks (in-context learning/modular/prompting based agents) wrap large language models with additional components—specialised vision encoders, hierarchical or search-based planners, episodic memory, and tool APIs—to tackle long-horizon tasks requiring perception, reasoning, and control [55, 1, 2, 20].

7 Conclusion

We presented OPENCUA, a comprehensive open-source framework addressing critical gaps in computer-use agent development. By offering annotation infrastructure, data processing pipelines, diverse datasets, effective training recipes, and ef cient evaluation benchmarks, we establish essential foundations for CUA research. Our models demonstrate strong performance across benchmarks while exhibiting clear data scaling laws and cross-domain generalization capabilities. By releasing all components—tools, datasets, code, and models—we aim to accelerate transparent CUA research, enabling the community to systematically investigate these agents' capabilities, limitations, and risks as they increasingly mediate our digital interactions and execute consequential decisions on our behalf.

8 Aknowledge

We thank Su Yu, Caiming Xiong, Binyuan Hui, and the anonymous reviewers for their insightful discussions and valuable feedback. We are grateful to Moonshot AI for providing training infrastructure and annotated data. We also sincerely appreciate Zaida Zhou, Zhengtao Wang, Flood Sung, Hao Hu, Huarong Chen, Yanxu Chen, Calvin, Qizheng Gu, and Dikang Du from the Kimi Team for their strong infrastructure support and helpful guidance. Finally, we extend our deepest thanks to all annotators for their tremendous effort and contributions to this project.

References

[1] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. arXiv preprint arXiv:2410.08164, 2024.

[2] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. arXiv preprint arXiv:2504.00906, 2025.

[3] Anthropic. Claude computer use. https://www.anthropic.com/news/3-5-models-and-computer-use, 2024. Accessed: 2025-05-03.

[4] Anthropic. Introducing claude 4, 5 2025. URL https://www.anthropic.com/news/claude-4. Accessed: 2025-07-28.

[5] Anthropic. Claude's extended thinking. https://www.anthropic.com/research/visible-extended-thinking, 2025. Accessed: 2025-05-03.

[6] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Ming-Hsuan Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report. CoRR, abs/2502.13923, 2025.

[7] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale, 2024. URL https://arxiv.org/abs/2409.08264.

[8] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Shuai Ren, and Hongsheng Li. Amex: Android multi-annotation expo dataset for mobile gui agents. arXiv preprint arXiv:2407.17490, 2024.

[9] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, et al. Guicourse: From general vision language models to versatile gui agents. arXiv preprint arXiv:2406.11317, 2024.

[10] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. Seeclick: Harnessing gui grounding for advanced visual gui agents. arXiv preprint arXiv:2401.10935, 2024.

[11] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL https://arxiv.org/abs/2406.13352.

[12] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems, 36:28091–28114, 2023.

[13] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vázquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? Forty- rst International Conference on Machine Learning, 2024. URL https://openreview.net/forum?id=BRfqYrikdo.

[14] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. arXiv preprint arXiv:2410.05243, 2024.

[15] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. CoRR, abs/2410.05243, 2024. URL https://doi.org/10.48550/arXiv.2410.05243.

[16] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In The Thirteenth International Conference on Learning Representations, 2025.

[17] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1. 5-vl technical report. arXiv preprint arXiv:2505.07062, 2025.

[18] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In European Conference on Computer Vision, pages 161–178. Springer, 2024.

[19] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 2024.

[20] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. Advances in Neural Information Processing Systems, 36:51991–52008, 2023.

[21] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. arXiv preprint arXiv:2504.07981, 2025.

[22] Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on computer control agents, 2024. URL https://arxiv.org/abs/2406.03679 .

[23] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. arXiv preprint arXiv:2411.17465, 2024.

[24] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. arXiv preprint arXiv:2406.08451, 2024.

[25] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. ArXiv preprint, 2021. URL https://arxiv.org/abs/2112.09332 .

[26] OBS Project. Obs studio. URL https://github.com/obsproject/obs-studio . Accessed 2025-08-04.

[27] OpenAdaptAI. Openadapt: Ai- rst process automation with large multimodal models (lmms). URL https://github.com/OpenAdaptAI/OpenAdapt . Accessed 2025-08-04.

[28] OpenAI. Operator, 2025. URL https://openai.com/research/operator .

[29] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. Ui-tars: Pioneering automated gui interaction with native agents, 2025. URL https://arxiv.org/abs/2501.12326 .

[30] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. Advances in Neural Information Processing Systems, 36:59708–59728, 2023.

[31] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. ArXiv, abs/2309.15817, 2023. URL https://api.semanticscholar.org/CorpusID:262944419 .

[32] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 2024.

[33] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, et al. Kimi-vl technical report. arXiv preprint arXiv:2504.07491, 2025.

[34] TheDuckAI. Ducktrack. URL https://github.com/TheDuckAI/DuckTrack . Accessed 2025-08-04.

[35] Bowen Wang, Xinyuan Wang, Jiaqi Deng, Tianbao Xie, Ryan Li, Yanzhe Zhang, Gavin Li, Toh Jing Hua, Ion Stoica, Wei-Lin Chiang, Diyi Yang, Yu Su, Yi Zhang, Zhiguo Wang, Victor Zhong, and Tao Yu. Computer agent arena: Compare test computer use agents on crowdsourced real-world tasks, 2025.

[36] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. ArXiv preprint, 2024. URL https://arxiv.org/abs/2409.12191 .

[37] Michael Wornow, Avanika Narayan, Ben Viggiano, Ishan S. Khare, Tathagat Verma, Tibor Thompson, Miguel Angel Fuentes Hernandez, Sudharsan Sundar, Chloe Trujillo, Krrish Chawla, Rongfei Lu, Justin Shen, Divya Nagaraj, Joshua Martinez, Vardhan Agrawal, Althea Hudson, Nigam H. Shah, and Christopher Re. Wonderbread: A benchmark for evaluating multimodal foundation models on business process management tasks, 2024. URL https://arxiv.org/abs/2406.13264 .

[38] Chen Henry Wu, Jing Yu Koh, Ruslan Salakhutdinov, Daniel Fried, and Aditi Raghunathan. Dissecting adversarial robustness of multimodal lm agents. In International Conference on Learning Representations, 2024. URL https://api.semanticscholar.org/CorpusID:270562791

[39] Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal llms, 2023. URL https://arxiv.org/abs/2312.14135 .

[40] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A foundation action model for generalist GUI agents. CoRR, abs/2410.23218, 2024. doi: 10.48550/ARXIV.2410.23218. URL https://doi.org/10.48550/arXiv.2410.23218 .

[41] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. Os-atlas: A foundation action model for generalist gui agents, 2024. URL https://arxiv.org/abs/2410.23218 .

[42] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.

[43] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. ArXiv preprint, 2024. URL https://arxiv.org/abs/2404.07972 .

[44] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025. URL https://arxiv.org/abs/2505.13227 .

[45] Tianbao Xie, Mengqi Yuan, Danyang Zhang, Xinzhuang Xiong, Zhennan Shen, Zilong Zhou, Xinyuan Wang, Yanxu Chen, Jiaqi Deng, Junda Chen, Bowen Wang, Haoyuan Wu, Jixuan Chen, Junli Wang, Dunjie Lu, Hao Hu, and Tao Yu. Introducing osworld-veri ed. xlang.ai, July 2025. URL https://xlang.ai/blog/osworld-verified .

[46] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials. arXiv preprint arXiv:2412.09605, 2024.

[47] Yiheng Xu, Hongjin Su, Chen Xing, Boyu Mi, Qian Liu, Weijia Shi, Binyuan Hui, Fan Zhou, Yitao Liu, Tianbao Xie, Zhoujun Cheng, Siheng Zhao, Lingpeng Kong, Bailin Wang, Caiming Xiong, and Tao Yu. Lemur: Harmonizing natural language and code for language agents. In International Conference on Learning Representations, 2024.

[48] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Uni ed pure vision agents for autonomous gui interaction. arXiv preprint arXiv:2412.04454, 2024.

[49] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents, 2025. URL https://arxiv.org/abs/2504.01382 .

[50] Zonghan Yang, Peng Li, Ming Yan, Ji Zhang, Fei Huang, and Yang Liu. React meets actre: When language agents enjoy training data autonomy. arXiv preprint arXiv:2403.14589, 2024.

[51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations (ICLR), 2023.

[52] Jiwen Zhang, Jihao Wu, Yihua Teng, Minghui Liao, Nuo Xu, Xiao Xiao, Zhongyu Wei, and Duyu Tang. Android in the zoo: Chain-of-action-thought for gui agents. ArXiv preprint, 2024. URL https://arxiv.org/abs/2403.02713 .

[53] Longtao Zheng, Zhiyuan Huang, Zhenghai Xue, Xinrun Wang, Bo An, and Shuicheng Yan. Agentstudio: A toolkit for building general virtual agents, 2025. URL https://arxiv.org/abs/2403.17918 .

[54] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, 2024.

[55] Wangchunshu Zhou, Yuchen Eleanor Jiang, Long Li, Jialong Wu, Tiannan Wang, Shi Qiu, Jintian Zhang, Jing Chen, Ruipu Wu, Shuai Wang, Shiding Zhu, Jiyu Chen, Wentao Zhang, Xiangru Tang, Ningyu Zhang, Huajun Chen, Peng Cui, and Mrinmaya Sachan. Agents: An open-source framework for autonomous language agents, 2023. URL https://arxiv.org/abs/2309.07870 .

Table of Contents in Appendix

A Limitations 21

B AGENTNETBENCH 21

C Dataset Statistics, Annotation Details, and AgentNetTool Details 23

C.1 AGENTNET Statistics and Analysis 23

C.1.1 Diversity 23

C.1.2 Complexity 24

C.1.3 Action distribution 24

C.2 Annotation Details: Annotation Strategy, Annotator Source and Cost 24

C.3 AGENTNET TOOL: System Design and Key Features 25

C.3.1 Tool Features 25

C.3.2 Privacy Protection 26

D OSWorld-Veri ed Results Detail 27

E Error Study 28

F OSWorld Case Example 29

G AgentNet Training Data Example 33

G.1 L1 Data Example 33

G.2 L2 Data Example 35

G.3 L3 Data Example 38

A Limitations

The scalability of AGENTNET dataset is inherently limited by human annotation efforts. Although AGENTNET TOOL streamlines the data collection process, expanding the dataset beyond its current size would require additional human resources. Exploring alternative data sources or semi-automated annotation methods could help address this limitation. Additionally, though OPEN CUA strives to collect authentic computer-use data from personal devices, our ethical requirement for explicit informed consent regarding data practices inevitably introduces selection bias. While our dataset maintains high diversity and authenticity, it necessarily excludes data from users who, upon understanding the potential risks, opt not to participate. This is a limitation we accept to uphold responsible data collection.

B AGENTNETBENCH

Figure 13: Illustration of the AgentNet Benchmark evaluation pipeline

There are several online benchmarks [42, 7] that evaluate agent performance in desktop environments. However, these online benchmarks typically require substantial computational resources for environment setup, making evaluations expensive, slow, and difficult to reproduce consistently through time due to their reliance on dynamic environments. Meanwhile, they only provide sparse, high-variance signals (i.e., trajectory-level accuracy). Another line of works, such as ComputerAgentArena [35], evaluate agent's performance of open-ended tasks on live environments through human preference. To address the limitations of online evaluation benchmarks, we introduce an offline CUA evaluation benchmark, AGENTNETBENCH, comprising 100 representative tasks selected from the AGENTNET dataset. Tasks were strategically chosen from the center of sub-domain clusters (as detailed in Section C.1.1), ensuring diversity and representativeness across applications and websites on Windows and macOS platforms. Each task was manually reviewed to refine goals and remove redundant actions. Notably, we manually provide multiple valid action options at each step because of the inherent multiplicity of valid actions in computer-use tasks.

Benchmark statistics and evaluation dimensions The AGENTNETBENCH maintains a balanced domain distribution consisting of 38 Work tasks, 29 Daily tasks, 24 Professional tasks, and 9 System & Web Setup tasks. The tasks are split between two operating systems, with 61 tasks from Windows and 39 tasks from macOS. Screen resolutions are categorized into three levels (high, medium, and low) as detailed in Table 6 (note that, for practical purposes, all images in the benchmark are resized from their original resolutions). The distribution of actions within these tasks and additional benchmark statistics are also presented comprehensively in Table 6.

Multiple action choices for enhanced accuracy Previous offline benchmarks [30, 22] typically define a single ground-truth action at each step. This practice can negatively impact accuracy by disregarding alternative valid choices that an agent may reasonably make in real-world interactions. In contrast, in AGENTNETBENCH, we annotate multiple plausible action choices for each step to better reflect real-world decision-making variability.

Step success rate calculation and action matching criteria To calculate the Step Success Rate (Step SR), we evaluate the correctness of agent actions at each individual step using precise matching

Table 6: Comprehensive Statistics of AgentNetBench

Domain Distribution		Operating System Distribution	
Work	38	Windows	61
Daily	29	macOS (Darwin)	39
Professional	24		
System & Web Setup	9		
Resolution Distribution		**Overall Statistics**	
High	20	Total Tasks	100
Medium	33	Avg. Steps/Task	17.63
Low	47	Total Actions	2143

Action Distribution					
click	850	(67.0%)	doubleClick	19	(1.5%)
rightClick	17	(1.3%)	press	28	(2.2%)
dragTo	27	(2.1%)	write	137	(10.8%)
moveTo	45	(3.5%)	hotkey	30	(2.3%)
scroll	18	(1.4%)	terminate	100	(7.6%)

criteria tailored to different action types. For coordinate-based actions (click , doubleClick , moveTo, dragTo, rightClick , and hscroll), we de ne bounding boxes around each action's target location; the agent earns the step success point if its predicted coordinates fall within these bounding boxes. For content- or keyboard-based actions, such as write, we measure correctness by computing the edit distance between the predicted and ground-truth text; actions like hotkey and press require perfect matches of the speci ed key combinations. For the scroll action, correctness depends on two key criteria: the agent's output coordinates must be within the designated bounding box, and the scrolling direction must exactly match the ground truth. Finally, the correctness of the terminate action depends on the agent appropriately terminating at precisely the correct step – neither prematurely nor delayed. Considering the distribution of actions (see Table 6), these ne-grained evaluation rules ensure accurate and fair evaluation of agent capabilities in diverse interaction scenarios.

AGENTNETBENCH strongly correlates with online benchmark performance The of ine benchmark primarily assesses an agent's decision-making capability by evaluating its rst-choice accuracy at each task step. While agents can leverage self-re ection to recover from errors made in earlier steps, of ine and online SRs should correlate under a low step budget. Figure 14 and Table 4 indeed demonstrate a clear positive correlation, speci cally following a power-law relation between the online task success rate (under a 15 step budget) and the of ine step success rate. Therefore, metrics obtained from our of- ine benchmark provide a reliable indicator of an agent's foundational pro ciency and its adaptability to realistic, resource-constrained online tasks.

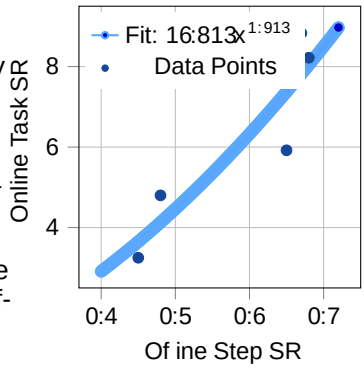

Figure 14: Of ine vs. Online evaluation.